

supported by the language. Lisp code, after it's read by the parser, is made of data structures that you can traverse.

If you understand how compilers work, you'll realise that what's really going on is not so much that Lisp has a strange syntax (parentheses everywhere!), but that it has no syntax. You write programs in the parse trees that get generated within the compiler when other languages are parsed. But these parse trees are fully accessible to your programs. You can write programs that manipulate them. In Lisp, these programs are called macros. They are programs that write programs. (If you ever were to enter 'The Matrix', you'd be happy that you were a Lisp maestro.)

We know that Java must be pretty good, because it is the cool programming language. Or is it? Within the hacker subculture, there is another language called Perl that is considered a lot cooler than Java. But there is yet another, Python, whose users tend to look down on Perl, and another called Ruby, that some see as the heir apparent to Python. If you look at these languages in order, Java, Perl, Python and Ruby, you'll notice an interesting pattern—at least, you'll notice this pattern if you are a Lisp hacker. Each one is progressively more like Lisp. Python copies even those features that many Lisp hackers consider to be mistakes. And if you'd shown people Ruby in 1975 and described it as a dialect of Lisp with syntax, no one would have argued with you.

Programming languages have almost caught up with 1958! Lisp was first discovered by John McCarthy in 1958, and popular programming languages are only now catching up with the ideas he developed back then.

Lisp enlightens you as an individual

All married men would relate to Steven Levy when he illustrates an example of how hackers think, in his book, 'Hackers: Heroes of the Computer Revolution'. Marge Saunders would drive back into the garage on a weekend morning, and would ask her husband, Bob, "Would you like to help me bring in the groceries?" He would reply, "No." Stunned, she would drag in the groceries herself. After the same thing occurred a few times, she exploded, hurling curses at him and demanding to know why he always said, "No" to her question.

"That's a stupid question to ask," he said. "Of course I won't like to help you bring in the groceries. If you ask me if I'll help you bring them in, that's another matter."

When I used to program in my favourite OO programming language, my response was no different. Luckily for me, I discovered Lisp. It gave me a holistic view of the self, the cosmos, and also taught me that there are better responses to a question than a simple Yes/No. From then on, I have learned that the right answer to a Marge Saunder-like question would be "Sure, dear! Do you need me to do anything else for you?" Needless to say, my

wife is a happier person, and we celebrated our seventh anniversary last month.

The functional programming edge

In his famous paper, 'Why Functional Programming Matters', computer scientist R John M Hughes says that conventional languages place conceptual limits on the way problems can be modularised. Functional languages push those limits back. Two features of functional languages in particular—higher-order functions and lazy evaluation, can contribute greatly to modularity. As an example, Lisp allows us to manipulate lists and trees, program several numerical algorithms, and implement the alpha-beta heuristic (an algorithm from Artificial Intelligence used in game-playing programs). Since modularity is the key to successful programming, functional languages like Lisp are vitally important to the real world.

Getting started

Any language that obeys the central principles of Lisp is considered a Lisp dialect. However, the vast majority of the Lisp community uses two Lisps: ANSI Common Lisp (often abbreviated CL) and Scheme. Here, I will be exclusively talking about the ANSI Common Lisp dialect, the slightly more popular of the two.

Many great Lisp compilers are available, but one in particular is the easiest to get started with: CLISP, an open source Common Lisp. It is simple to install, and runs on any operating system—Windows platforms, Macs, and Linux variants. Mac users may want to consider LispWorks, which will be easier to get running on their machines.

Installing CLISP

You can download a CLISP installer from <http://clisp.cons.org/>. On Windows, you simply run an installed program. On a Mac, there are some additional steps, which are detailed on the website. On a Debian-based Linux machine, you should find that CLISP already exists in your standard sources. Just run `apt-get install clisp` at the command line and you'll have CLISP installed automatically. For other Linux distributions (Fedora, SuSE, etc), you can use the standard packages listed under 'Linux packages' on the CLISP website.

Starting it up

To start CLISP, run `clisp` at your command line. If all goes according to plan, you'll see the prompt shown in Figure 1. Like all Common Lisp environments, CLISP will automatically place you into a *read-eval-print-loop* (REPL) after you start it up. This means that you can immediately start typing in Lisp code. Try it out by typing `(* 7 (+ 4 3))`. You'll see the result printed below the expression:

```
> (* 7 (+ 4 3))
49
```